Demo App Projects

Project 1: Banking Demo Apps

Customer-facing apps

Desktop Web Banking Portal
Mobile Web Banking Portal
Native Mobile Banking App

Employee-facing app

Windows Desktop Banking Employee App

Demo story

A customer does normal banking tasks from web or mobile.
A bank employee handles internal review, support, or approval work from the desktop app.

Example functionality

Customer side:

Login
View accounts
View transactions
Transfer money
Pay a bill
Freeze/unfreeze card
Update profile
Send support message

Employee desktop side:

Search customer
Review account activity
Approve or reject transfer
Review support message
Update customer status
Export account/transaction report

Project 2: Car Insurance Demo Apps

Customer-facing apps

Desktop Web Insurance Portal
Mobile Web Insurance Portal
Native Mobile Insurance App

Employee-facing app

Windows Desktop Claims/Agent App

Demo story

A customer gets a quote, manages a policy, or files a claim from web/mobile.
An insurance employee reviews the claim or policy from the desktop app.

Example functionality

Customer side:

Get quote
Buy/view policy
Update vehicle information
Update driver information
File claim
Upload accident photos/documents
Track claim status
Make payment

Employee desktop side:

Search policyholder
Review claim
Approve/reject claim
Request more information
Update claim status
Generate claim summary

Project 3: Health Tech Demo Apps - Thiago

Customer-facing apps

Desktop Web Patient Portal
Mobile Web Patient Portal
Native Mobile Patient App

Employee-facing app

Windows Desktop Clinic/Admin App

Demo story

A patient books appointments, fills forms, views results, or sends requests from web/mobile. Clinic staff handles scheduling, patient records, refill requests, or intake review from the desktop app.

Example functionality

Patient side:

Book appointment
Fill intake form
View lab results
Update insurance information
Message provider
Request prescription refill
View visit summary

Employee desktop side:

Search patient
Review appointment queue
Review intake form
Approve/reject refill request
Update visit status
Export visit summary

Project 4: E-commerce Demo Apps - Deepansh

Customer-facing apps

Desktop Web Store
Mobile Web Store
Native Mobile Shopping App

Employee-facing app

Windows Desktop Order/Admin App

Demo story

A shopper browses, buys, tracks, or returns items from web/mobile.
An employee processes orders, inventory, refunds, or fulfillment from the desktop app.

Example functionality

Customer side:

Browse products
Search/filter products
View product details
Add to cart
Apply coupon
Checkout
Track order
Request return/refund

Employee desktop side:

Search order
Update fulfillment status
Process refund
Update inventory
Generate invoice
Export order report

The New Simple Rule

Use this rule for every domain:

User Type	App Type	Platforms
Customer / patient / shopper / policyholder	Customer-facing app	Desktop Web, Mobile Web, Native Mobile
Employee / agent / admin / staff	Internal app	Windows Desktop

So the final structure is still **4 projects**, but each project now has a cleaner platform split:

Project	Customer Apps	Employee App
Banking	Web, Mobile Web, Native Mobile	Windows Desktop
Car Insurance	Web, Mobile Web, Native Mobile	Windows Desktop
Health Tech	Web, Mobile Web, Native Mobile	Windows Desktop
E-commerce	Web, Mobile Web, Native Mobile	Windows Desktop

This is a lot more believable and easier to explain to the team.

Dummy Demo App Build Rules

Main Rule

These apps are not production applications.

They are demo shell apps built to show realistic user workflows for test automation demos. The goal is to emulate real business behavior, not to recreate real banking, insurance, health tech, or e-commerce systems.

If something does not help the demo, do not build it.

The app should feel realistic from the outside and stay simple on the inside.

A demo user should think:

“This looks like a real workflow.”

A developer should think:

“This is just mock data and simple screens.”

---

1. Login Rules

Do not build real authentication.

Do not build:

Real user registration
Password reset infrastructure
Encrypted password storage
OAuth
SSO
Real account lockout systems
Complex session management
User permission databases

Build this instead:

Use a simple hardcoded list of demo users.

Example:

Role	Username	Password
Customer	customer.demo	Demo123
Employee	employee.demo	Demo123
Manager	manager.demo	Demo123
Admin	admin.demo	Demo123

The app only needs to check whether the entered username and password match one of the demo users.

Rule

Login should feel real to the demo audience, but it should be fake behind the scenes.

---

2. MFA Rules

Do not build production MFA.

Do not build:

Real Microsoft Authenticator dependency
Real Google Authenticator dependency
Real push approval

Real SMS provider
Real user enrollment system
Complex MFA recovery flows
Production-grade security logic

Build this instead:

Use simple, controlled MFA-style flows that help show testRigor capabilities.

Allowed MFA options:

Fixed MFA code
Role-based MFA code
Email-based MFA code using testRigor-controlled email
QR-code MFA using a known test data secret

Examples:

Fixed code: 123456
Customer code: 111111
Employee code: 222222
Admin code: 333333

QR-code MFA rule

QR-code MFA is allowed only when it is demo-controlled.

Requirements:

The QR code must be predictable.
The secret must be available in test data.
The test must be able to get or generate the code reliably.
The flow must not require a human to manually use an authenticator app.
The app must not depend on a paid or external service.

Best use cases:

Banking
Health Tech
Insurance, if security is part of the demo story

Usually unnecessary for:

E-commerce, unless the demo specifically focuses on account security

Rule

MFA is allowed when it is free, stable, demo-controlled, and useful for showing testRigor capabilities.

The goal is not to prove the app has real security. The goal is to show that testRigor can handle MFA-style workflows.

---

3. Email Rules

Do not integrate with external email systems.

Do not use:

Gmail integration
Outlook integration
SendGrid
Twilio
Mailchimp
Customer email systems
Employee-owned inboxes
Paid email providers

Build this instead:

Email notifications are allowed if they use testRigor's built-in email system and domain.

Allowed email flows:

Verification code email
Order confirmation email
Claim confirmation email
Appointment confirmation email
Password reset-style email
Support ticket confirmation email
Payment confirmation email

Email content should be useful for validation.

Examples:

Confirmation number
One-time code
Order total
Appointment date
Claim status
Support ticket number
Payment amount
Policy number

Rule

Email is allowed when it uses testRigor-controlled email and does not require external integrations.

The goal is to show that testRigor can validate email-based workflows without adding external system complexity.

---

4. Data Rules

Do not build complex databases.

Do not build:

Full database schemas
User account tables
Real customer records
Real transaction systems
Complex backend services
Paid cloud databases
Production-like data pipelines

Build this instead:

Use one of these free or simple options:

Hardcoded JSON files
Static mock data

Local storage
In-memory data
SQLite
Free-tier Firebase or Supabase only if truly needed

Rule

Use the cheapest data storage that supports the demo workflow.

If static JSON works, use static JSON.

---

5. Payment Rules

Do not build real payments.

Do not integrate with:

Stripe
PayPal
Credit card processors
Bank payment rails
ACH
Insurance billing systems
Real invoices

Build this instead:

Use fake payment forms and fake card numbers.

Example:

Card number: `4111 1111 1111 1111`
Expiration: `12/30`
CVV: `123`

After submission, show a success or failure message.

Rule

The app should demonstrate checkout or payment flow behavior, not process money.

---

6. File Upload Rules

Do not build real document storage unless necessary.

Do not build:

Cloud file storage
Virus scanning
S3 buckets
Document lifecycle systems
Real OCR
Real image analysis

Build this instead:

Allow the user to select a file and then display:

File name
Upload status
Preview placeholder
Success message

Example:

“accident-photo.jpg uploaded successfully.”

Rule

File upload only needs to prove that the workflow accepts a file and reacts correctly.

---

7. Search Rules

Do not build advanced search engines.

Do not build:

Elasticsearch
Full-text indexing
AI search
Complex ranking
Real filtering infrastructure

Build this instead:

Search against a small static list.

Example searchable items:

Customers
Orders
Claims
Transactions
Appointments
Products
Policies
Patients

Rule

Search should return predictable demo results every time.

---

8. Reports and Exports Rules

Do not build real reporting engines.

Do not build:

Analytics dashboards
BI integrations
Complex charting systems
Real scheduled reports
Large CSV generation

Build this instead:

Simple report pages with fake numbers.

Optional export behavior:

Generate a small CSV
Generate a simple PDF
Show a fake “Report downloaded” confirmation

Rule

Reports only need to show that testRigor can validate a report page, export action, or confirmation message.

---

9. Admin Approval Rules

Do not build real workflow engines.

Do not build:

Complex approval chains
Dynamic permissions
Escalation rules
Audit log systems
Queue routing logic

Build this instead:

Use a simple status change.

Example statuses:

Pending
Approved
Rejected
Needs More Information

Employee clicks Approve or Reject.
The status changes on the screen.

Rule

Approval workflows should be simple, visible, and easy to reset.

---

10. Role and Permission Rules

Do not build a full permission system.

Do not build:

Custom role management
Permission matrices
Dynamic access policies
User group management

Build this instead:

Hardcode a few roles.

Example roles:

Customer
Employee
Manager
Admin

Each role sees a slightly different menu or page.

Rule

Roles should support demo scenarios, not enterprise security.

---

11. Business Logic Rules

Do not build real business calculations.

Do not build:

Real loan underwriting
Real insurance risk scoring
Real medical decision logic
Real tax calculation
Real shipping rate logic
Real fraud detection

Build this instead:

Use simple fake rules.

Examples:

Bank transfer over $5,000 requires review.
Insurance claim over $2,000 starts as Pending Review.
Appointment cannot be booked if the time slot is unavailable.
Coupon `DEMO10` gives 10% off.
Order over $100 qualifies for free shipping.
A refill request starts as Pending Review.

Rule

Business logic should be simple enough that a demo audience understands it immediately.

---

12. API Rules

Do not depend on external APIs.

Avoid:

Paid APIs
Unstable third-party APIs

Real banking APIs
Real insurance APIs
Real medical APIs
Real shipping APIs
Real payment APIs

Build this instead:

Use mock APIs or local endpoints.

Examples:

```
/api/users
/api/orders
/api/claims
/api/appointments
/api/patients
/api/policies
/api/reset-demo-data
```

Rule

The demo should never fail because an outside service is down.

---

13. Data Reset Rules

Do not make testers manually clean up data.

Do not require someone to manually reset:

Orders
Claims
Appointments
Transactions
Account statuses
Cart data
Uploaded files

Policy statuses
Patient requests

Build this instead:

Each app should have a simple reset option.

Examples:

Hidden “Reset Demo Data” button
Reset endpoint
Admin reset screen
Automatic reset on app restart

Rule

Every demo app must be easy to return to a known starting state.

---

14. Error Handling Rules

Do not build every possible error case.

Do not build hundreds of validations.

Build this instead:

Create a small number of useful demo errors.

Examples:

Invalid login
Missing required field
Invalid payment card
Insufficient funds
Claim missing document
Appointment time unavailable
Product out of stock
Invalid promo code

Invalid MFA code
Expired verification code

Rule

Only build errors that create useful test scenarios.

---

15. UI Rules

Do not over-design the app.

Do not spend time on:

Perfect branding
Advanced animations
Complicated layouts
Highly customized components
Production-level design systems

Build this instead:

Clean, simple, realistic screens.

The UI should be:

Easy to understand
Stable for automation
Consistent across apps
Not ugly, but not overbuilt
Predictable for demos

Rule

The app should look believable, not expensive.

---

16. Native Mobile Rules

Do not rebuild a completely different product for native mobile.

The native mobile app should support the same core customer workflows as the web and mobile web app.

Build this instead:

A simple mobile app with:

Login
Home/dashboard
Main customer workflow
Confirmation screen
Basic profile/settings

Rule

Native mobile should show platform coverage, not a separate business system.

---

17. Mobile Web Rules

Do not create a separate mobile web product.

Mobile web should usually be the responsive version of the desktop web customer app.

Build this instead:

One web app that works well on desktop and mobile browser sizes.

Rule

Desktop web and mobile web should share as much as possible.

---

18. Windows Desktop Rules

Do not build customer-facing Windows desktop apps.

Windows desktop should be for employee/internal workflows.

Examples:

Bank employee reviews transaction
Insurance agent reviews claim
Clinic admin reviews appointment
Warehouse employee updates order

Rule

Windows desktop apps should represent internal employee systems, not customer portals.

---

19. Security Rules

Do not build real security.

Do not handle:

Real personal data
Real medical data
Real financial data
Real passwords
Real tokens
Real customer records

Build this instead:

Use fake data only.

Example demo people:

Maria Lopez
Jordan Miller
Emma Johnson
Chris Brown
Alex Carter
Priya Shah
Taylor Reed
Riley Smith

Rule

Everything must be fake and safe to show in a public demo.

---

20. Integration Rules

Do not build real integrations unless they are required for a testRigor demo capability.

Avoid integrating with:

Real payment systems
Real email providers
Real SMS providers
Real CRM systems
Real EHR systems
Real banking systems
Real insurance platforms
Real shipping providers
Real authentication providers

Allowed exception

A simple integration-like flow is allowed when it is:

Free
Stable
Demo-controlled
Easy to reset

Useful for showing testRigor capabilities
Not dependent on external paid systems

Examples:

testRigor-controlled email validation
Mock API responses
QR-code MFA with a known secret
Fake payment confirmation
Fake document upload confirmation

Rule

If the integration can be mocked, mock it.

---

21. Scope Control Rule

Before building anything, ask:

Does this help us demo testRigor?

If yes, build the simplest believable version.

If no, do not build it.

Final Build Philosophy

Do not build the real system.

Build the smallest believable version of the system that lets testRigor demonstrate the workflow.

The apps should be realistic enough for demos, but simple enough to build and maintain cheaply.